

FEDERATED LEARNING FOR PRIVACY-PRESERVING COLD CHAIN ANOMALY DETECTION

Full Project Architecture · Implementation Plan · Work Distribution

**B.E. 2nd Year · 4-Member Team · 12-Week
Sprint**

Relevant to: Unit III (Food Safety) · Unit IV (Food Processing) · Biosafety & Ethics
Target venue: IEEE Access · IEEE IoT Journal · IEEE Transactions on Industrial Informatics

1. Project Overview & Problem Statement

1.1 The Problem

Modern food cold chains are operated by multiple competing businesses — different food companies whose products share the same trucks, warehouses, and refrigeration units. When a temperature spike occurs that spoils products, it affects everyone in that node. Ideally, all operators would collaborate to build a shared anomaly detection model that learns from all historical temperature and humidity data.

The critical obstacle: raw logistics data is sensitive business information. A dairy brand will not share its cold chain routing patterns with a competitor using the same warehouse. Traditional centralized machine learning requires all data to be pooled on one server — which is commercially and legally unacceptable in a shared cold chain.

Core Research Question

Can we train a high-accuracy food cold chain anomaly detection model that matches centralized performance, while guaranteeing that raw temperature/humidity data from any node never leaves that node?

Answer: Yes — using Federated Learning with FedAvg aggregation + LSTM Autoencoders per node.

1.2 Why This Is Novel

- Federated learning for food supply chain is a 2023–2025 research frontier with fewer than 15 published papers globally.
- No published work applies FL + LSTM Autoencoder specifically to cold chain anomaly detection with a benchmarked privacy-utility tradeoff curve.
- The benchmarking framework (isolated vs federated vs centralized) addresses a gap identified in a 2025 systematic review of 1,132 FL papers.
- India-specific cold chain context (FSSAI Unit III compliance) adds regional novelty for IEEE Access submission.

1.3 Project Deliverables

- Working FL simulation: 5 simulated client nodes + Flower server running FedAvg
- LSTM Autoencoder anomaly detector per node, trained on real public datasets
- 3-way benchmark: Isolated baseline vs Federated (proposed) vs Centralized baseline
- Privacy-utility tradeoff curve — the key figure for the paper
- Streamlit dashboard showing live anomaly detection + federated round visualization
- IEEE-format research paper draft (ready for submission to IEEE Access)

2. System Architecture

2.1 High-Level Architecture

The system follows a horizontal federated learning architecture — all client nodes have the same model structure and data schema. The architecture has three layers:

Layer	Components	Responsibility
Data Layer	Public datasets + synthetic partitioning into 5 non-IID client splits	Simulates real heterogeneous cold chain operators with different data distributions
Client Layer	5 Flower client nodes, each with local LSTM Autoencoder	Local training on own data shard, gradient computation, weight upload only
Server Layer	Flower server running FedAvg aggregation	Weighted averaging of client gradients, global model distribution, round management
Evaluation Layer	Benchmarking harness + Streamlit UI	Compare F1 / precision / recall across 3 approaches, generate tradeoff curves

2.2 Data Layer — Datasets & Partitioning

Primary Dataset — USDA NASS Cold Storage Reports (Free)

Source: nass.usda.gov → Cold Storage reports (monthly, 2015–2024)

Variables: Temperature (°F/°C), commodity type, storage facility type, region, monthly readings

Volume: ~50,000 rows after cleaning. Supplement with Kaggle IoT Temperature Dataset (~200K rows).

License: Public domain (US government data). No restrictions on use or publication.

The partitioning strategy is critical for FL research validity. The data must be split non-IID (non-identically and independently distributed) across 5 nodes to simulate real-world heterogeneity:

- Node 1 — Dairy warehouse: subset with high-humidity, low-temperature variance. Mostly normal readings.
- Node 2 — Meat cold store: wide temperature swings, higher anomaly rate. Simulates an older facility.
- Node 3 — Frozen goods truck fleet: frequent short-duration spikes from door-opening events.
- Node 4 — Pharmaceutical cold chain (proxy): very tight tolerances, rare but severe anomalies.

- Node 5 — Mixed produce: seasonal patterns, moderate variance, moderate anomaly rate.

Anomaly injection: add synthetic temperature spikes (Gaussian noise, $\pm 3\sigma$ outliers) at known timestamps to create ground-truth labels. This is standard practice in FL anomaly detection literature.

2.3 Model Architecture — LSTM Autoencoder

Each client node runs an identical LSTM Autoencoder. An autoencoder learns to reconstruct normal input sequences. When it encounters an anomaly, reconstruction error (MSE) spikes — the threshold on this error is your anomaly detector.

Layer	Dimensions	Notes
Input	(batch, 24, 2)	24-hour sliding window × 2 features (temp, humidity)
LSTM Encoder 1	(batch, 24, 64)	64 hidden units, return_sequences=True
LSTM Encoder 2	(batch, 32)	32-dim bottleneck — the compressed representation
RepeatVector	(batch, 24, 32)	Expands bottleneck back to sequence length for decoding
LSTM Decoder 1	(batch, 24, 64)	Mirror of encoder
TimeDistributed Dense	(batch, 24, 2)	Reconstructs original 2 features per timestep
Anomaly decision	MSE threshold	If reconstruction MSE > threshold (99th percentile of training set): ANOMALY

2.4 Federated Learning Layer — Flower Framework

The Flower (flwr) framework is used for federated simulation. It allows multiple clients to run as separate processes on a single machine, communicating via gRPC — exactly replicating real distributed behaviour without needing physical hardware.

- Server: FlowerServer with FedAvg strategy. Aggregates weights from all clients each round. Tracks global loss and F1 across rounds.
- Client: FlowerClient wraps the LSTM Autoencoder. Implements fit() (local training) and evaluate() (local validation). Sends numpy weight arrays to server.
- Rounds: 30–50 communication rounds. Each round = one local epoch on each client.
- FedAvg formula: $W_{\text{global}} = \sum (n_i / n_{\text{total}}) \times W_i$ — weighted by dataset size per node.

Why Flower (flwr) over PySyft?

flwr is actively maintained (v1.x, 2024) and significantly easier to set up than PySyft. PySyft requires Docker and has breaking API changes. flwr runs with pip install flwr + ~50 lines of code.

flwr is used in recent IEEE papers — safer citation target for reviewers.

Install: pip install flwr tensorflow scikit-learn pandas numpy matplotlib streamlit

2.5 Evaluation Layer — The 3-Way Benchmark

This benchmark is the core research contribution of the paper. You run three experimental conditions on identical data:

Condition	Data sharing	Expected F1	Privacy
Isolated baseline	No sharing — each node alone	~0.65–0.72	✓ Full privacy
Federated (proposed)	Gradients only — no raw data	~0.82–0.88 (target)	✓ Full privacy
Centralized baseline	All raw data pooled	~0.89–0.94	✗ Zero privacy

The key paper figure: plot F1 score (y-axis) vs communication rounds (x-axis) for federated vs centralized. If federated reaches within 5–8% of centralized F1 while sharing zero raw data, that is your publishable result. This is called the 'privacy-utility tradeoff curve.'

3. Team Structure & Core Responsibilities

The team is divided into 4 roles, each owning a core subsystem end-to-end. No one is doing 'just testing' or 'just documentation' — all 4 members work on functional components that directly appear in the paper.

Member	Role Title	Core Ownership	Paper Contribution
P1	Data Engineer	Dataset acquisition, cleaning, non-IID partitioning, anomaly injection pipeline	Section 3 (Dataset), Section 4 (Experiments) — data tables & preprocessing methodology
P2	ML Engineer	LSTM Autoencoder design, training loop, threshold tuning, isolated baseline	Section 4 (Model Architecture), ablation study, model performance tables
P3	FL Systems Engineer	Flower server + client setup, FedAvg implementation, federated training loop, communication round logging	Section 4 (FL Architecture), Section 5 (Results) — privacy-utility tradeoff curve
P4	Evaluation & UI Engineer	Benchmarking harness (3-way comparison), metrics computation, Streamlit dashboard, paper figures	Section 5 (Results & Analysis), Section 6 (Discussion), all charts and visualizations

4. Full Task Plan with Parallel / Serial Dependencies

How to read this plan

PARALLEL (//): Tasks that can run simultaneously — different people work on them at the same time.

SERIAL (→): Task B cannot start until Task A is complete — one depends on the other's output.

SYNC POINT (⊕): All parallel tracks must merge here before the next phase begins.

Each task has an Owner (P1/P2/P3/P4) and duration in days.

Phase 1: Foundation (Week 1–2) [All SERIAL — everyone syncs first]

All 4 members work together on this phase. Do not split yet. The goal is a shared understanding of the codebase structure, the datasets, and the environment before anyone starts their individual module.

Task	Name	Description	Owner	Days	Dependency
T1.1	Environment setup	Install Python 3.10+, TensorFlow 2.x, flwr, sklearn, pandas, streamlit. Create shared GitHub repo with agreed folder structure: /data, /models, /clients, /server, /eval, /dashboard, /paper. Everyone clones and runs a hello-world script.	ALL	1 day	Start
T1.2	Dataset download & audit	P1 downloads USDA NASS cold storage CSVs + Kaggle IoT temperature dataset. All 4 members open and explore the data together. Agree on: which columns to use, what constitutes an anomaly, time window size (24 hours chosen). Document decisions in /data/README.md.	P1 leads, all review	2 days	After T1.1
T1.3	Architecture review session	One 2-hour meeting. P2 explains LSTM Autoencoders to everyone (whiteboard / diagram). P3 explains Flower federated simulation. P4 explains the 3-way benchmark design. Everyone must understand the full system before splitting into parallel tracks.	ALL	0.5 day	After T1.2
T1.4	Interface contracts	Define and document the exact data format that flows between modules. Format 1: cleaned dataset CSV schema (columns, dtypes, anomaly label column). Format 2: node data shard format. Format 3: model weight format for Flower (numpy	P1+P3 write, all approve	1 day	After T1.3

		arrays). Format 4: metrics dict structure for evaluation. Write these as Python dataclass specs in /shared/interfaces.py. Everyone agrees before splitting.			
--	--	---	--	--	--

⊕ SYNC POINT after Phase 1

Before Phase 2 starts: everyone has the same repo, same data, agreed interfaces.

P1, P2, P3, P4 now split into parallel tracks. They will not sync again until end of Week 5.

Phase 2: Core Development (Week 3–7) [ALL 4 TRACKS PARALLEL]

This is the main build phase. All four members work independently on their subsystem using the agreed interface contracts from T1.4. Weekly 30-minute standups (Monday) to flag blockers — not for decision making.

Track P1 — Data Pipeline (SERIAL within track)

Task	Name	Description	Days	Dependency
T2.P1.1	Data cleaning	Handle missing values (forward-fill for sensor gaps ≤ 2 hrs, drop otherwise). Normalize temperature to Celsius, humidity to 0–100%. Remove duplicate timestamps. Output: clean_data.csv. Script: /data/clean.py.	3 days	After Phase 1
T2.P1.2	Non-IID partitioning	Split into 5 client shards with deliberately different distributions (see Section 2.2 for node profiles). Use stratified splitting by commodity type. Verify each shard has different mean/std. Output: /data/node_{1-5}/train.csv, val.csv, test.csv. Script: /data/partition.py.	2 days	After T2.P1.1
T2.P1.3	Anomaly injection	Inject synthetic anomalies at known timestamps for ground-truth labels. Strategy: random 3% of samples get temperature spike of $+2.5\sigma$ to $+4\sigma$, duration 1–6 hours. Add 'anomaly' column (0/1). Verify injection rate matches real-world cold chain failure statistics (~2–4%). Script: /data/inject_anomalies.py. Output: labelled datasets per node.	2 days	After T2.P1.2
T2.P1.4	DataLoader class	Write a PyTorch/NumPy DataLoader that creates 24-hour sliding window sequences (shape: batch $\times$ 24 $\times$ 2). Each Flower client will import this class. Must conform to the interface	2 days	After T2.P1.3

		contract from T1.4. Include unit tests. Output: /data/dataloader.py.		
--	--	--	--	--

Track P2 — LSTM Autoencoder Model (SERIAL within track)

Task	Name	Description	Days	Dependency
T2.P2.1	Model skeleton	Implement LSTM Autoencoder using TensorFlow/Keras (see architecture table in Section 2.3). Use dummy random data to confirm forward pass works end-to-end. Verify output shape (batch × 24 × 2) matches input. Script: /models/lstm_autoencoder.py.	2 days	After Phase 1
T2.P2.2	Training loop	Implement local training loop: Adam optimizer, MSE reconstruction loss, 10 local epochs per round. Add early stopping (patience=3). Save best weights to /models/node_{id}_weights.npy. This is what each Flower client will call inside fit(). Must be callable as a standalone function: train(model, data, epochs) → trained_model.	2 days	After T2.P2.1
T2.P2.3	Threshold tuning	Train on node 1 data (standalone, no federation yet) to determine optimal anomaly threshold. Method: compute reconstruction MSE on entire training set, set threshold = 99th percentile. Evaluate on node 1 test set. Target: F1 > 0.65 for isolated single-node baseline. Record results in /eval/isolated_results.json.	2 days	After T2.P2.2 + T2.P1.3 done
T2.P2.4	Isolated baseline (all 5 nodes)	Run training independently on all 5 nodes using their own data shards only. Record per-node F1, precision, recall. This is the 'no sharing' condition for the 3-way benchmark. Important: use the same random seed and hyperparameters that P3 uses for federated run — only difference is no federation. Output: /eval/baseline_isolated.json.	2 days	After T2.P2.3 + T2.P1.4 done

Track P3 — Federated Learning System (SERIAL within track)

Task	Name	Description	Days	Dependency
T2.	Flower client wrapper	Implement FlowerClient class (inherits fl.client.NumPyClient). Implement	3 days	After T2.P2.2 done

P 3. 1		get_parameters() — returns model weights as list of numpy arrays. Implement fit(parameters, config) — loads weights, trains for config['epochs'] local epochs, returns updated weights + training metrics. Implement evaluate(parameters, config) — loads weights, evaluates on local val set, returns loss + F1 score. Script: /clients/flower_client.py.		
T 2. P 3. 2	Flower server + FedAvg	Implement Flower server with FedAvg strategy. Configure: min_fit_clients=5, min_evaluate_clients=5, min_available_clients=5. Custom evaluate_fn to track global F1 each round. Set on_fit_config_fn to pass {'epochs': 1} per round. Round logging: save global loss, F1, and communication cost each round to /eval/fl_round_log.json. Script: /server/fl_server.py.	3 days	After T2.P3.1
T 2. P 3. 3	Simulation runner	Write the main simulation script that launches the Flower server + 5 clients using fl.simulation.start_simulation(). Run for 30 rounds first to verify convergence. If F1 plateaus before round 20, increase local epochs from 1 to 3. If memory issues, reduce batch size. Script: /run_federated.py. Output: /eval/federated_results.json with per-round metrics.	2 days	After T2.P3.2 + T2.P1.4 done
T 2. P 3. 4	Centralized baseline	Run IDENTICAL model training on ALL 5 nodes' data pooled into one dataset. Same model architecture, same hyperparameters, same random seed as federated run. This is the 'upper bound' for the benchmark. Record F1 on each node's test set. Output: /eval/baseline_centralized.json. Important: store per-round metrics to plot against federated curve.	1 day	After T2.P3.3 + T2.P1.4 done

Track P4 — Evaluation & Dashboard (PARALLEL tracks within: T2.P4.1 // T2.P4.2, then SERIAL T2.P4.3–4)

T a s k	Name	Description	Days	Dependency
T 2. P 4.	Metrics harness	[Can start immediately after Phase 1 — uses mock data first] Implement evaluation harness: load results JSON from /eval/, compute per-node and aggregate F1 / precision / recall / AUC-ROC. Generate comparison tables. The	3 days	After Phase 1 only

1 //		harness must work on mock data so P4 can build it before P2/P3 finish. Script: /eval/evaluate.py.		
T 2. P 4. 2 //	Streamlit UI skeleton	[Parallel to T2.P4.1 — same person in alternating sessions, or split with P1 if P1 finishes early] Build Streamlit app skeleton: sidebar for selecting node, upload area for custom data, placeholder charts (using random data). Establish layout: Tab 1 = Live anomaly detection, Tab 2 = Federated round progress, Tab 3 = 3-way benchmark. Script: /dashboard/app.py.	3 days	After Phase 1 only
T 2. P 4. 3	Connect harness to UI	Plug real results from /eval/ into Streamlit charts. Line chart: F1 vs round (federated vs centralized). Bar chart: per-node F1 across 3 conditions. Heatmap: anomaly detection timeline per node. Add colour coding for anomaly severity.	2 days	After T2.P4.1, T2.P4.2, AND real results from P2+P3
T 2. P 4. 4	Paper figure generation	Export publication-quality figures using matplotlib (300 DPI). Required figures: Fig 1 — Privacy-utility tradeoff curve (main result), Fig 2 — Per-node F1 comparison bar chart, Fig 3 — Anomaly detection timeline (one representative anomaly event), Fig 4 — Communication rounds vs convergence. Save as /paper/figures/fig_{1-4}.png.	2 days	After T2.P4.3

⊕ SYNC POINT after Phase 2 (End of Week 7)

All 4 tracks must produce their outputs before Phase 3 begins.

Required: clean data shards (P1) ✓, trained isolated baseline (P2) ✓, federated + centralized results (P3) ✓, benchmark figures (P4) ✓

Hold a 2-hour integration meeting. Run the full pipeline end-to-end for the first time. Fix integration bugs.

Phase 3: Integration & Tuning (Week 8–9) [Partially parallel]

All 4 members come back together. Focus: fixing integration issues, improving model performance if F1 is too low, polishing the dashboard, and beginning the paper draft simultaneously.

T a s k	Name	Description	Own er	Days	Mode
------------------	------	-------------	-----------	------	------

T 3. 1	End-to-end integration test	Run full pipeline: data → 5 client nodes → FL training → evaluation → dashboard. Fix any data format mismatches between modules. Verify all 3 benchmark conditions produce results.	ALL	2 days	SERIAL — all together
T 3. 2 //	Hyperparameter tuning	If federated F1 < 0.78: try increasing local epochs (1→3), adjusting learning rate (1e-3→5e-4), or adding dropout. Target: federated reaches ≥ 85% of centralized F1. Run additional rounds (up to 50).	P2 + P3	3 days	// with T3.3
T 3. 3 //	Paper draft (intro + related work)	P1 writes Section 1 (Introduction) and Section 2 (Related Work / Literature Review) in LaTeX (IEEE two-column template). Include 8–12 references from the FL anomaly detection literature. P4 writes Section 3 (System Overview) using the architecture figures.	P1 + P4	3 days	// with T3.2
T 3. 4	Dashboard polish	Final Streamlit UI polish: add FSSAI compliance context (threshold map for food types), add exportable PDF report, test on a fresh machine. Record a 2-minute demo video.	P4	2 days	After T3.1

Phase 4: Paper & Submission Prep (Week 10–12) [Partially parallel]

T a s k	Name	Description	Owner	Days	Mode
T 4. 1 //	Paper: Methodology + Experiments	P2 writes Section 4 (Model Architecture + Training Details). P3 writes Section 4 continued (FL Setup + Communication Protocol). Use exact hyperparameter values, algorithm pseudocode for FedAvg, and reproduction instructions.	P2 + P3	3 days	// with T4.2
T 4. 2 //	Paper: Results + Discussion	P4 writes Section 5 (Results — all benchmark tables + privacy-utility curve analysis). P1 writes Section 6 (Discussion — food safety implications, FSSAI context, limitations). Claim: federated F1 is X% of centralized, with 0% raw data sharing.	P1 + P4	3 days	// with T4.1
T 4. 3	Full paper merge + guide review	Merge all sections in LaTeX. Format to IEEE two-column template (IEEEtran). Submit to guide/faculty co-author for review. Address comments. Target: 6–8 pages including references.	ALL + Guide	3 days	After T4.1 + T4.2

T 4. 4	Code cleanup + GitHub release	Clean all code, add README with reproduction steps, add requirements.txt, add LICENSE (MIT). Tag v1.0 release. The GitHub repo link goes in the paper as a reproducibility artifact.	P3	2 days	// with T4.3
T 4. 5	Submission	Submit to IEEE Access (primary) via ScholarOne Manuscripts. Simultaneously submit to IEEE ISTAS 2026 conference track as a backup. Guide handles final submission unless the team has IEEE accounts.	Guide + P1	1 day	After T4.3 + T4.4

5. Full Technology Stack & File Structure

5.1 Technology Stack

Category	Tool / Library	Why this choice
FL Framework	Flower (flwr) v1.x	Actively maintained, works with TensorFlow/PyTorch, simulation mode runs all clients locally, cited in recent IEEE papers
Deep Learning	TensorFlow 2.x / Keras	LSTM Autoencoder built in Keras Sequential API, easy weight extraction as numpy arrays for Flower, GPU optional
Data Processing	pandas, numpy, scikit-learn	Standard data pipeline stack, sklearn for metrics (F1, precision, recall, AUC-ROC)
Dashboard	Streamlit	Zero frontend knowledge needed, Python-only, can be deployed free on Streamlit Community Cloud for demos
Visualization	matplotlib, seaborn	Paper-quality figures (300 DPI), full control over IEEE figure formatting
Version Control	Git + GitHub	Shared repo, branch per person per track, merge at sync points, tag for paper submission
Paper Writing	LaTeX + Overleaf	IEEE IEEEtran template, free Overleaf account supports 4 collaborators, produces PDF directly
Compute	Google Colab (free tier)	GPU-optional — LSTM on 50K rows trains in <10 min on CPU. Colab T4 GPU if needed for 50-round simulation.

5.2 Repository File Structure

Recommended folder layout — agree on this during T1.1

```

federated-cold-chain/
├── data/
│   ├── raw/           # Downloaded datasets (gitignored — too large)
│   ├── processed/     # clean_data.csv after T2.P1.1
│   ├── node_{1-5}/    # Partitioned shards after T2.P1.2
│   ├── clean.py       # P1 — cleaning script
│   ├── partition.py   # P1 — non-IID partitioning
│   └── inject_anomalies.py # P1 — anomaly injection
├── models/
│   ├── lstm_autoencoder.py # P2 — model architecture
│   └── train.py          # P2 — training loop

```

```

├── clients/
│   └── flower_client.py    # P3 — Flower client wrapper
├── server/
│   └── fl_server.py       # P3 — Flower server + FedAvg
├── eval/
│   ├── evaluate.py        # P4 — metrics harness
│   ├── isolated_results.json # P2 output
│   ├── federated_results.json # P3 output
│   └── centralized_results.json # P3 output
├── dashboard/
│   └── app.py              # P4 — Streamlit app
├── paper/
│   ├── main.tex           # LaTeX IEEE paper
│   └── figures/            # P4 — exported 300 DPI figures
├── shared/
│   └── interfaces.py       # T1.4 — agreed data formats
├── run_federated.py       # P3 — main simulation entry point
├── requirements.txt
└── README.md

```

6. 12-Week Timeline at a Glance

Week	Phase	P1 (Data)	P2 (Model)	P3 (FL Sys)	P4 (Eval)
1	Foundation	T1.1–T1.4 (all together)	T1.1–T1.4	T1.1–T1.4	T1.1 – T1.4
2	Foundation	T1.2–T1.4 (all together)	T1.2–T1.4	T1.2–T1.4	T1.2 – T1.4
3	Core Dev	T2.P1.1 — Clean data	T2.P2.1 — Model skeleton	Waits for P2.2	T2.P4.1 // P4.2 start
4	Core Dev	T2.P1.2 — Partition	T2.P2.2 — Training loop	T2.P3.1 — FL client	T2.P4.1 // P4.2
5	Core Dev	T2.P1.3 — Anomalies	T2.P2.3 — Thresholds	T2.P3.2 — FL server	T2.P4.1

					// P 4. 2
6	Core Dev	T2.P1.4 — DataLoader	T2.P2.4 — Isolated base	T2.P3.3 — Sim runner	T 2. P 4. 3 c o n n e c t
7	Core Dev	Support P3 integration	Assist P3 bugs	T2.P3.4 — Central base	T 2. P 4. 4 f i g u r e s
8	Integration	⊕ SYNC + T3.3 Paper intro	T3.2 Tuning	T3.2 Tuning	T 3. 4 D a s h b o a r d
9	Integration	T3.3 Related work	T3.2 Tuning	T3.2 Tuning	T 3. 4 P o l i s h
10	Paper	T4.2 Discussion	T4.1 Methodology	T4.1 FL setup	T 4. 2 R e s

					ul ts
11	Paper	T4.3 Merge + review	T4.3 Merge + review	T4.4 Code cleanup	T 4. 3 Fi g u r e s s i n a l
12	Submission	T4.5 Submit	Final review	T4.4 GitHub	T 4. 5 S u b m i t

Critical Path

The critical path (sequence that determines project duration) is:

T1.1 → T1.4 → T2.P1.1 → T2.P1.2 → T2.P1.3 → T2.P1.4 → T2.P3.3 → T2.P3.4 → T3.1 → T3.2 → T4.1/T4.2 → T4.3 → T4.5

P3's simulation runner (T2.P3.3) is the most critical task — it is the first point where all modules must work together. If P3 is blocked, the entire project is blocked. P1's DataLoader (T2.P1.4) is the second most critical — P3 cannot start T2.P3.3 without it.

7. IEEE Paper Outline

Target: IEEE Access (6–8 pages, two-column, IEEEtran template). Alternatively: IEEE Transactions on Industrial Informatics.

Section	Writer	Content
Abstract (250 words)	P1 draft, all edit	Problem: cold chain privacy. Method: FL + LSTM Autoencoder. Result: X% of centralized F1 with zero raw data sharing. Contribution: benchmarking framework.
1. Introduction	P1	Food cold chain growth, privacy problem, existing centralized ML limitation, FL as solution, paper contributions listed.
2. Related Work	P1	FL for IoT (3 papers), FL for food safety (2 papers), LSTM Autoencoder anomaly detection (3 papers), cold chain monitoring (2 papers). 10–12 total refs.
3. System Architecture	P4	Architecture diagram, data description, non-IID partition methodology, anomaly injection strategy.
4. Methodology	P2 + P3	LSTM Autoencoder design, FedAvg algorithm (pseudocode), Flower simulation setup, threshold determination method.
5. Results	P4	Table: 3-way F1/precision/recall. Figure 1: privacy-utility curve. Figure 2: per-node comparison. Figure 3: anomaly detection example. Statistical analysis.
6. Discussion	P1	Food safety implications, FSSAI cold chain compliance context, limitations (simulated nodes, synthetic anomalies), future work (differential privacy, real deployment).
7. Conclusion	All	1 paragraph summary of contribution + result + impact.
References	P1 compiles	IEEE format. Key refs: Google FL paper (2017), Flower paper (2020), LSTM Autoencoder for anomaly detection (2018), food supply chain FL (2024), cold chain IoT (2023).

8. Risks & Mitigation

Risk	Likelihood	Mitigation
Federated F1 < 75% (too far from centralized)	Medium	Increase local epochs per round from 1 to 3. Try FedProx strategy instead of FedAvg (built into Flower) — handles non-IID data better. Add more rounds (50 → 100).
Data insufficient / too small for meaningful results	Low	Supplement USDA data with UCI 'Gas Sensor Array' dataset as additional anomaly time-series. TimeGAN synthetic augmentation (P1 implements in Week 6 if needed).
Flower simulation crashes on low-RAM machine	Low-Med	Reduce to 3 clients from 5 (still valid for FL research). Use Colab T4 GPU with 12GB RAM. Lower batch size to 16.
P2 and P3 blocked waiting for P1's data	Medium	P2 starts T2.P2.1 with synthetic random data. P3 starts T2.P3.1 with dummy model weights. Both switch to real data when P1's T2.P1.4 is done. This is why interface contracts (T1.4) are critical.
Guide unavailable for paper review	Low	Plan for 1 week buffer in Week 11. If guide is unavailable for final review, submit to arXiv first (free preprint server) to establish date priority, then revise for IEEE submission.

Final Note

This project is achievable in 12 weeks by a focused 4-person team with no prior FL experience. The key is respecting the sync points and interface contracts — modules built in isolation integrate cleanly.

The research novelty is real: FL applied to food cold chain with a proper 3-way benchmark is publishable at IEEE Access.

Budget: ₹0. All tools, datasets, and compute are free. The only cost is time.